

Elmenus QA Automation & Process Improvement

1. Automation (When & what to Automate)

Automation on a food delivery platform is not one decision, it is a sequence of investments, each unlocking the next. Based on my experience shipping QA on production systems, I would propose a tiered rollout rather than a single "big bang" automation effort.

Every tier below assumes the same constraint: **scope** is bounded by **team budget, knowledge**, and the realistic ROI of each layer. Automating everything sounds good in slideware; in practice, an over-ambitious framework that no one maintains costs more than the manual testing it replaced.

1.1 Recommended order of automation

Tier	Scope	Tooling	Why this order
1. API regression suite	Auth, menu fetch, cart, order creation, payment callback handling to run after every release	RestAssured + TestNG/JUnit, integrated with the release pipeline	Fastest to author, most stable, catches contract regressions before reaching UI
2. E2E happy-path monitor	Full order flow (browse → cart → checkout → pay → track) as a daily synthetic monitor on production	Playwright (web) + Appium (mobile) on a real-device cloud	Catches integration breaks no API test or unit test can — the chain itself is what matters
3. UI regression for stable flows	Stable, low-churn UI tests: login, address management, promo entry, profile screens	Same Playwright/Appium framework, extended page-object model	Highest manual effort to repeat each cycle; automating these frees the team for exploratory work

1.2 Tier 1 — API regression suite (start here)

Why API first: the API layer is where contracts live, and contracts are what break silently between releases. A passing UI test on a broken contract is a false positive waiting to embarrass the team. RestAssured gives us assertions that are fast, deterministic, and CI-friendly, and the suite can be wired into the release pipeline so no build promotes staging without a green API run.

What to cover first: the four endpoints (homepage feed, sign-in, search, add-to-cart — plus order creation and payment callback). These are the money paths. Negative cases (invalid auth, malformed payloads, idempotency on retry) matter as much as the happy paths.

Framework choices: RestAssured + TestNG or JUnit, with environment configs externalized so the same suite runs against staging and against a read-only subset on production. **Reports via Allure** for visibility. Schema validation built in from day one — when the contract changes, the test fails loudly rather than silently passing.

1.3 Tier 2 — End-to-end production heartbeat

Once the API suite is stable and trusted, the next step is a small set of end-to-end flows running as a daily synthetic monitor against production.

Tooling fit: Playwright for web, stable selectors, good network interception. Appium for mobile, running against a real-device cloud (BrowserStack or similar). Both fit the existing CI infrastructure.

1.4 Tier 3 — Stable UI flows

Once Tiers 1 and 2 are paying for themselves, automate the stable, low-churn UI flows that currently consume the most manual regression time: login, address management, promo entry, profile and notification settings. These do not need to run every build. The goal here is not coverage for its own sake; it is freeing the manual testers for exploratory work, where human judgment outperforms any script.

What NOT to automate yet: anything that changes weekly (the menu UI, restaurant cards, promotional banners), anything visual (those go to a visual regression tool, not Selenium-style assertions), and anything where the cost of writing the test exceeds the cost of running it manually for the next quarter.
ROI must be measured per test, not per suite.

2. QA Process Improvement

2.1 A note from prior experience — UX is not a lower-priority bug class

In my previous role working on a CRM system, I learned a lesson that directly applies here: **UX issues are often classified as "low severity" because nothing is functionally broken**, yet they are the bugs that quietly damage adoption, increase support load, and erode trust. A user who is confused, frustrated, or uncertain at checkout will abandon the order even though every system worked perfectly.

In food delivery, this risk is multiplied. The market is crowded; **Talabat, Mrsool, HungerStation, Jahez, and Elmenus** are all one swipe away on the same home screen. Users don't tolerate friction; they switch apps. A delayed cart icon, an unresponsive language toggle, or a confusing error message isn't just a UX flaw — it's a direct handoff to a competitor. Unlike enterprise software, there is no switching cost, no learning curve, no contract holding the user in place. The only thing keeping them in our app is the experience itself.

That's why I treat UX issues in this domain as first-class quality concerns, not cosmetic ones, So I recommend:

- **Weight UX bugs higher than the traditional severity scale suggests.** A misleading error message at checkout is not P3 — it is P1 if it can cost an order. The bug report (Bug-03 on the cart counter) reflects this it is technically "works as coded" but materially damage trust.
- **Involve QA in design reviews, not just at the end.** On the CRM work, the highest-leverage moment to fix a UX bug was before it was built. A 15-minute review of a Figma flow caught problems that would have taken days to find and re-test after development. The same logic applies here — QA should sit in on flow reviews for checkout, tracking, and any payment-related changes.
- **Track UX-only bugs separately in the defect dashboard.** Functional bug counts trend toward zero as a release stabilizes; UX bugs often hide in the long tail. Giving them their own dashboard prevents them from being deprioritized into oblivion.
- **Run small qualitative checks alongside the test plan.** Even five minutes of observing a non-tester completes the order flow surfaces friction points no test case will catch. On the CRM, this single practice changed how the team prioritized polish work.

None of this replaces the test plan, the API suite, or the production test described above. It complements them. The point is that on a consumer product like **Elmenus**, where the user has many alternatives one tap away, UX quality is not a soft metric, it is a retention metric, and QA should treat it accordingly.

2.2 Performance Testing for Peak-Traffic Seasons

The second improvement I'd recommend is establishing a **continuous performance testing practice**, not just functional testing. Food delivery traffic is not flat; it spikes hard and predictably:

- **Summer vacation**
- **Ramadan**
- **Weekends and holidays**
- **Marketing campaigns and promo codes**

In these windows, the system isn't tested by QA, it's tested by real customers. And if loading times slow down, payment APIs time out, or the order-tracking service lags, the impact is immediate:

abandoned carts, duplicate orders, frustrated drivers, and angry restaurants. Worse, in a market this competitive, a user who waits 8 seconds for the menu to load will simply switch to Talabat or Mrsool and may not come back even after the issue is fixed.

To prevent this, I'd recommend introducing the following into the QA cycle:

- **Load testing**
- **Stress testing**
- **Soak testing**
- **Spike testing**
- **API performance monitoring** with clear SLAs on critical endpoints (login, menu, cart, checkout, payment, order tracking)

Tools like **JMeter**, or **k6** can be integrated into the pre-release pipeline so performance regressions are caught *before* a launch, not during a Friday night peak.

The goal is simple: **the system must perform on its worst day, not its average day.** In food delivery, the difference between handling a traffic spike gracefully and degrading for ten minutes can mean thousands of lost orders, and a measurable shift of customers to competitors.